

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA5115

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

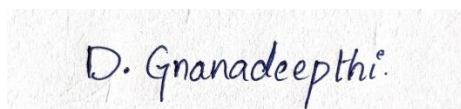
PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "D. Gnanadeepthi".

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.
 2. Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.
 3. Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction.
 4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.
 5. Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.
-

ANSWERS

1. Pseudocode: Simplified TLS Handshake

ALGORITHM Simplified_TLS_Handshake

BEGIN

1. CLIENT HELLO

Client → Server:

Send ClientHello

Send supported TLS versions

Send supported cipher suites

Generate and send ClientRandom

2. SERVER HELLO

Server → Client:

Receive ClientHello

Select TLS version and cipher suite

Generate ServerRandom

Send ServerHello

3. SERVER AUTHENTICATION

Server → Client:

Send Digital Certificate

Certificate contains:

- Server identity
- Server public key
- Certificate Authority (CA) signature

Client:

Verify certificate

Check certificate validity and CA signature

IF certificate is invalid THEN

Terminate connection

END IF

4. SESSION KEY GENERATION

Client and Server:

Perform key exchange using the server's public key

Generate a shared secret

Derive a symmetric session key from:

$\text{ClientRandom} + \text{ServerRandom} + \text{SharedSecret}$

5. HANDSHAKE VERIFICATION

Client → Server:

Send Finished message encrypted using session key

Server:

Verify Finished message

Server → Client:

Send Finished message encrypted using session key

Client:

Verify Finished message

6. SECURE DATA EXCHANGE

Client:

Encrypt application data using session key

Send encrypted data to Server

Server:

Decrypt data using session key

Process the data

Encrypt response using session key

Send encrypted response to Client

Client:

Decrypt the response using session key

7. TERMINATION

When communication is complete:

Client or Server sends TLS connection termination message

Close secure connection

END

Explanation:

The simplified TLS handshake establishes a secure communication channel between a client and server. The client first sends a ClientHello, and the server responds with a ServerHello and its digital certificate. The client verifies the certificate to authenticate the server. After authentication, both sides perform a key exchange and derive a common symmetric session key. This session key is then used to encrypt and decrypt application data, providing confidentiality and secure communication. Files, images, and data analysis are unavailable until usage resets at 3:53 PM. Continue chatting with text only, or upgrade for more access.

2. Pseudocode: Firewall Rule Engine

ALGORITHM Firewall_Rule_Engine

BEGIN

1. Define firewall rules:

Rule 1: Allow TCP from trusted IP addresses on port 80

Rule 2: Allow TCP on port 443

Rule 3: Block traffic from blacklisted IP addresses

Rule 4: Block all other traffic

2. FOR each incoming packet DO

Read packet details:

Source_IP

Destination_IP

Source_Port

Destination_Port

Protocol

Decision ← "BLOCK"

FOR each firewall rule DO

IF packet matches rule THEN

IF rule action = "ALLOW" THEN

Decision ← "ALLOW"

ELSE IF rule action = "BLOCK" THEN

Decision $\leftarrow$ "BLOCK"

END IF

BREAK

END IF

END FOR

IF Decision = "ALLOW" THEN

Forward packet to destination

Log "Packet ALLOWED"

ELSE

Drop packet

Log "Packet BLOCKED"

END IF

END FOR

END

Rule Matching Function

FUNCTION Match_Rule(Packet, Rule)

```
BEGIN

IF Rule.Source_IP ≠ "ANY" AND

    Packet.Source_IP ≠ Rule.Source_IP THEN

        RETURN FALSE

END IF


IF Rule.Destination_Port ≠ "ANY" AND

    Packet.Destination_Port ≠ Rule.Destination_Port THEN

        RETURN FALSE

END IF


IF Rule.Protocol ≠ "ANY" AND

    Packet.Protocol ≠ Rule.Protocol THEN

        RETURN FALSE

END IF


RETURN TRUE


END
```

Working:

The firewall examines every incoming packet and extracts its source IP address, destination port, and protocol. It compares these values with the predefined firewall rules. If a matching ALLOW rule is found, the packet is forwarded; if a BLOCK rule is found, the packet is dropped. If no rule matches, the default action is BLOCK, providing better security.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient